ISITCOM
Institut Superieur d'Informatique et des Technologies de Communication

Framework Frontend II

Seance 4 : Authentification, Middleware & Performance

Securisation et Optimisation de l'Application E-Commerce

Master 1 SWM - 5h - Annee 2025-2026

Enseignant : Abdelweheb GUEDDES

abdelweheb.gueddes@isitcom.tn

Theorie (1h30) - Pratique guidee (3h30) - Projet E-Commerce

Objectifs Pedagogiques

A l'issue de cette seance, l'etudiant sera capable de :

- Comprendre les concepts d'**authentification** et d'**autorisation** dans le web moderne.
- Installer et configurer **NextAuth.js** pour gerer l'authentification dans Next.js.
- Implementer une authentification complete avec **email/mot de passe** et **OAuth** (Google, GitHub).
- Creer et utiliser un **Middleware Next.js** pour proteger les routes sensibles.
- Mettre en place un systeme de **roles** (USER, ADMIN) avec controle d'acces.
- Optimiser les performances de l'application avec **next/image**, **next/font** et le **lazy loading**.
- Analyser et ameliorer le score **Lighthouse** de l'application.

Prerequis

Prerequis indispensables :

- Avoir suivi les seances 1, 2 et 3 (App Router, Data Fetching, Backend avec Prisma).
- Avoir un projet e-commerce fonctionnel avec base PostgreSQL et CRUD produits.
- Comprendre les concepts de sessions et cookies.

Prerequis techniques :

- Node.js LTS installe.
- PostgreSQL local ou Docker fonctionnel.
- Projet e-commerce des seances precedentes.

Concept Fondamental

Fil rouge E-commerce (suite du projet) : Dans cette seance, nous allons securiser notre boutique en ligne :

- → Inscription et connexion des utilisateurs
- → Panier lie a l'utilisateur connecte
- → Dashboard admin reserve aux administrateurs
- → Optimisation des images produits et des performances globales

Table des matières

1 Partie 1 – Theorie (1h30) 4
1.1 1. Introduction a l'Authentification Web 4
1.2 2. NextAuth.js : la solution pour Next.js 4
1.3 3. Middleware Next.js : protection des routes 6
1.4 4. Gestion des Roles (RBAC) 7
1.5 5. Optimisation des Performances 8

2 Partie 2 – Pratique Guidee (3h30) 9
2.1 Exercice 1 : Installation et Configuration de NextAuth.js (45 min) 9
2.2 Exercice 2 : Pages d'Inscription et de Connexion (45 min) 17
2.3 Exercice 3 : Middleware de Protection des Routes (30 min) 32
2.4 Exercice 4 : Dashboard Admin avec Verification des Roles (45 min) 36
2.5 Exercice 5 : Optimisation des Performances (60 min) 44
2.6 Exercice 6 : Page Panier Complete (30 min) 57

3 Recapitulatif et Structure Finale du Projet 61

4 Livrable de la Seance 63

5 Conclusion 64

1 Partie 1 – Theorie (1h30)

1.1 1. Introduction a l'Authentification Web

Concept Fondamental

Authentification vs Autorisation : deux concepts distincts
Authentification (AuthN) : Verification de l'identite de l'utilisateur.

- Qui etes-vous ?
- Prouver son identite (mot de passe, empreinte, token...)
- Exemples : connexion email/mot de passe, OAuth Google

Autorisation (AuthZ) : Verification des permissions de l'utilisateur.

- Que pouvez-vous faire ?
- Controler l'acces aux ressources selon le role
- Exemples : admin peut supprimer, user peut seulement lire

Explication Detaillee

Les differentes methodes d'authentification :

1. **Sessions (cote serveur)**
   - Le serveur stocke l'etat de connexion
   - Un cookie contient l'ID de session
   - Traditionnel, simple a implementer
   - Probleme : scalabilite (sessions en memoire)
2. **JWT (JSON Web Token)**
   - Token signe contenant les informations utilisateur
   - Stocke cote client (cookie ou localStorage)
   - Stateless : pas de stockage serveur
   - Ideal pour les API et microservices
3. **OAuth 2.0**
   - Delegation d'authentification a un tiers (Google, GitHub, Facebook)
   - L'utilisateur s'authentifie chez le provider
   - Notre application recoit un token d'acces
   - Avantage : pas de mot de passe a gerer

1.2 2. NextAuth.js : la solution pour Next.js

Concept Fondamental

Pourquoi NextAuth.js ?
NextAuth.js (maintenant appele **Auth.js**) est la solution d'authentification officiellement recommandee pour Next.js.
Avantages :

- ✔ **Multi-providers** : Google, GitHub, Facebook, Twitter, email...
- ✔ **Sessions securisees** : cookies HttpOnly, CSRF protection
- ✔ **Integration Prisma** : stockage des utilisateurs en base
- ✔ **JWT ou Database sessions** : au choix
- ✔ **Callbacks** : personnalisation complete du flux
- ✔ **TypeScript** : types inclus

Architecture NextAuth.js :

1. L'utilisateur clique sur "Se connecter"
2. Redirection vers le provider (Google) ou formulaire local
3. Le provider renvoie les informations utilisateur
4. NextAuth cree une session et un cookie
5. L'application peut verifier la session a tout moment

⚖ Comparaison Detaillee

Sessions JWT vs Sessions Database

Critere	**JWT (strategy : "jwt")**	**Database (strategy : "database")**
Stockage	Token dans cookie	Session en base
Performance	Rapide (pas de requete)	Requete DB a chaque verification
Revocation	Difficile	Facile (supprimer en base)
Taille cookie	Plus grand (contient data)	Petit (juste ID)
Scalabilite	Excellente	Depend de la DB
Notre choix	**JWT** pour la performance	

1.3 3. Middleware Next.js : protection des routes

Concept Fondamental

Qu'est-ce qu'un Middleware ?
Un **Middleware** est du code qui s'execute **avant** que la requete atteigne la page.
Dans Next.js, le middleware :

- S'execute a chaque requete entrante
- Peut rediriger, modifier les headers, bloquer l'acces
- Fonctionne a l'Edge (tres rapide)
- Fichier unique : `middleware.js` a la racine

Cas d'utilisation :

- Rediriger les utilisateurs non connectes
- Verifier les roles (admin, user)
- Ajouter des headers de securite
- Internationalisation (redirection selon la langue)

Explication Detaillee

Fonctionnement du Middleware :

```
Requete HTTP
     |
     v
+------------------+
|   Middleware     |  <-- Verification ici !
+------------------+
     |
     | (si autorise)
     v
+------------------+
|     Route        |
+------------------+
     |
     v
   Reponse
```

Important : Le middleware s'execute AVANT le rendu de la page. Si l'utilisateur n'est pas authentifie, il est redirige AVANT de voir la page protegee.

1.4 4. Gestion des Roles (RBAC)

Concept Fondamental

RBAC : Role-Based Access Control
Le controle d'acces base sur les roles (RBAC) permet de definir des permissions selon le role de l'utilisateur.

Roles dans notre e-commerce :

USER : Client standard

- Voir les produits
- Gerer son panier
- Passer des commandes
- Voir son historique

ADMIN : Administrateur

- Tout ce que USER peut faire
- CRUD sur les produits
- CRUD sur les categories
- Voir toutes les commandes
- Gerer les utilisateurs

Implementation :

1. Champ `role` dans la table User (deja present en seance 3)
2. Verification du role dans le Middleware
3. Verification du role dans les Server Actions
4. Affichage conditionnel dans l'interface

1.5 5. Optimisation des Performances

Performance et Optimisation

Pourquoi optimiser ?
Les performances impactent directement :

- **SEO** : Google penalise les sites lents
- **Conversion** : -1s de chargement = +7% de conversions
- **Experience utilisateur** : abandon si > 3 secondes
- **Mobile** : connexions plus lentes

Outils d'optimisation Next.js :

1. **next/image** : Optimisation automatique des images
   - Redimensionnement automatique
   - Formats modernes (WebP, AVIF)
   - Lazy loading natif
   - Placeholders flous
2. **next/font** : Optimisation des polices
   - Pas de FOUT (Flash of Unstyled Text)
   - Hebergement local automatique
   - Preconnect automatique
3. **Lazy loading** : Chargement differe
   - `next/dynamic` pour les composants lourds
   - Chargement a la demande
   - Reduction du bundle initial

Explication Detaillee

Lighthouse : l'outil de mesure
Lighthouse est l'outil de Google pour mesurer les performances web.
Metriques principales :

- **FCP (First Contentful Paint)** : Premier rendu visible
- **LCP (Largest Contentful Paint)** : Plus gros element rendu
- **CLS (Cumulative Layout Shift)** : Stabilite visuelle
- **TTI (Time to Interactive)** : Temps avant interaction
- **TBT (Total Blocking Time)** : Temps de blocage

Objectifs :

- ✔ Performance : > 90
- ✔ Accessibility : > 90
- ✔ Best Practices : > 90
- ✔ SEO : > 90

2 Partie 2 – Pratique Guidee (3h30)

Concept Fondamental

Objectif pratique : Securiser et optimiser notre application e-commerce :

- ✔ Authentification complete avec NextAuth.js
- ✔ Protection des routes avec Middleware
- ✔ Dashboard admin avec verification des roles
- ✔ Optimisation des images et des performances

Livrable fin de seance : Application e-commerce securisee avec authentification, roles et performances optimisees.

2.1 Exercice 1 : Installation et Configuration de NextAuth.js (45 min)

Exercice 1 : Installation NextAuth.js (45 min)

But : Installer NextAuth.js et configurer l'authentification de base.

Etape 1 – Installer les dependances

```
# Installation de NextAuth.js et bcrypt pour le hashage des mots de
    passe
npm install next-auth bcryptjs
```

Explication Detaillee

Packages installes :

- **next-auth** : Bibliotheque d'authentification pour Next.js
- **bcryptjs** : Hashage securise des mots de passe

Etape 2 – Mettre a jour le schema Prisma

Correction Complete

Modifiez le fichier `prisma/schema.prisma` pour ajouter les champs necessaires a NextAuth.

```
/*
  prisma/schema.prisma
  Schema e-commerce avec authentification NextAuth.js

  Nouveaux champs pour User :
  - password : mot de passe hashe
  - emailVerified : date de verification email
  - image : avatar utilisateur
```

```
  Nouvelles tables pour NextAuth :
  - Account : comptes OAuth lies
  - Session : sessions actives
  - VerificationToken : tokens de verification
*/

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Table User mise a jour pour NextAuth
model User {
  id            String    @id @default(cuid())
  email         String    @unique
  name          String?
  password      String?   // Mot de passe hashe (null si OAuth)
  emailVerified DateTime? // Date verification email
  image         String?   // URL avatar
  role          String    @default("USER") // "USER" ou "ADMIN"
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  // Relations NextAuth
  accounts      Account[]
  sessions      Session[]

  // Relations E-commerce
  orders        Order[]
  cartItems     CartItem[]
}

// Table Account pour OAuth (Google, GitHub...)
model Account {
  id                String  @id @default(cuid())
  userId            String
  type              String
  provider          String
  providerAccountId String
  refresh_token     String?
  access_token      String?
  expires_at        Int?
  token_type        String?
  scope             String?
  id_token          String?
  session_state     String?

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([provider, providerAccountId])
}

```

```
// Table Session pour les sessions database (optionnel)
model Session {
  id           String   @id @default(cuid())
  sessionToken String   @unique
  userId       String
  expires      DateTime

  user         User     @relation(fields: [userId], references: [id], onDelete: Cascade)
}

// Table VerificationToken pour la verification email
model VerificationToken {
  identifier String
  token      String   @unique
  expires    DateTime

  @@unique([identifier, token])
}

// Les autres tables restent identiques (Category, Product, CartItem, Order, OrderItem)
model Category {
  id        String   @id @default(cuid())
  name      String   @unique
  slug      String   @unique
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  products  Product[]
}

model Product {
  id          String   @id @default(cuid())
  title       String
  slug        String   @unique
  description String
  price       Float
  stock       Int      @default(0)
  imageUrl    String?
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  categoryId  String?
  category    Category? @relation(fields: [categoryId], references: [id])

  cartItems   CartItem[]
  orderItems  OrderItem[]
}

model CartItem {
  id        String   @id @default(cuid())
  quantity  Int      @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

```

```
  userId    String
  user      User     @relation(fields: [userId], references: [id])

  productId String
  product   Product  @relation(fields: [productId], references: [id])

  @@unique([userId, productId])
}

model Order {
  id        String     @id @default(cuid())
  status    String     @default("PENDING")
  total     Float       @default(0)
  createdAt DateTime    @default(now())
  updatedAt DateTime    @updatedAt

  userId    String
  user      User       @relation(fields: [userId], references: [id])

  items     OrderItem[]
}

model OrderItem {
  id        String   @id @default(cuid())
  title     String
  price     Float
  quantity  Int      @default(1)

  orderId   String
  order     Order    @relation(fields: [orderId], references: [id])

  productId String?
  product   Product? @relation(fields: [productId], references: [id])
}
```

Etape 3 – Appliquer la migration

```
# Creer et appliquer la migration
npx prisma migrate dev --name add_nextauth_tables

# Regenerer le client Prisma
npx prisma generate
```

Resultat Attendu

Apres cette commande, Prisma aura cree les tables Account, Session et VerificationToken, et mis a jour la table User avec les nouveaux champs.

Etape 4 – Creer les variables d'environnement

Correction Complete

Ajoutez ces variables dans votre fichier `.env` :

```
# .env

# Base de donnees (existant)
DATABASE_URL="postgresql://isitcom:isitcom123@localhost:5432/
    isitcom_shop"

# NextAuth.js
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="votre-secret-tres-long-et-aleatoire-ici"

# OAuth Google (optionnel pour commencer)
GOOGLE_CLIENT_ID="votre-google-client-id"
GOOGLE_CLIENT_SECRET="votre-google-client-secret"

# OAuth GitHub (optionnel pour commencer)
GITHUB_ID="votre-github-client-id"
GITHUB_SECRET="votre-github-client-secret"
```

Astuce Professionnelle

Generer un secret securise :

```
# Sur Linux/Mac
openssl rand -base64 32

# Sur Windows PowerShell
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -
    Max 256 }))
```

Etape 5 – Creer la configuration NextAuth

Correction Complete

Creez le fichier `src/lib/auth.js` pour centraliser la configuration :

```
/*
  src/lib/auth.js
  Configuration centrale de NextAuth.js

  Ce fichier definit :
  - Les providers d'authentification (Credentials, Google, GitHub)
  - Les callbacks pour personnaliser les sessions
  - Les pages personnalisees (login, erreur)
*/

import CredentialsProvider from "next-auth/providers/credentials";
import GoogleProvider from "next-auth/providers/google";
import GitHubProvider from "next-auth/providers/github";
import { prisma } from "@/lib/prisma";
import bcrypt from "bcryptjs";

// Configuration des options NextAuth
export const authOptions = {
  // Providers d'authentification
  providers: [
    // Provider Credentials : email/mot de passe
    CredentialsProvider({
      name: "credentials",
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Mot de passe", type: "password" }
      },

      // Fonction de verification des identifiants
      async authorize(credentials) {
        // Verification que les champs sont remplis
        if (!credentials?.email || !credentials?.password) {
          throw new Error("Email et mot de passe requis");
        }

        // Recherche de l'utilisateur en base
        const user = await prisma.user.findUnique({
          where: { email: credentials.email }
        });

        // Verification que l'utilisateur existe
        if (!user) {
          throw new Error("Utilisateur non trouve");
        }

        // Verification que l'utilisateur a un mot de passe (pas OAuth)
        if (!user.password) {
          throw new Error("Connectez-vous avec Google ou GitHub");
        }

```

```
        // Verification du mot de passe
        const passwordMatch = await bcrypt.compare(
          credentials.password,
          user.password
        );

        if (!passwordMatch) {
          throw new Error("Mot de passe incorrect");
        }

        // Retourne l'utilisateur (sera stocke dans le token)
        return {
          id: user.id,
          email: user.email,
          name: user.name,
          role: user.role,
          image: user.image
        };
      }
    }),

    // Provider Google (optionnel)
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID || "",
      clientSecret: process.env.GOOGLE_CLIENT_SECRET || ""
    }),

    // Provider GitHub (optionnel)
    GitHubProvider({
      clientId: process.env.GITHUB_ID || "",
      clientSecret: process.env.GITHUB_SECRET || ""
    })
  ],

  // Callbacks pour personnaliser le comportement
  callbacks: {
    // Callback JWT : ajoute des infos au token
    async jwt({ token, user }) {
      // Si l'utilisateur vient de se connecter
      if (user) {
        token.id = user.id;
        token.role = user.role;
      }
      return token;
    },

    // Callback session : expose les infos dans la session client
    async session({ session, token }) {
      // Ajoute l'id et le role a la session
      if (session.user) {
        session.user.id = token.id;
        session.user.role = token.role;
      }
      return session;
    },

    // Callback signIn : logique supplementaire a la connexion OAuth
    async signIn({ user, account }) {
```

```
      // Pour OAuth : creer ou mettre a jour l'utilisateur
      if (account?.provider === "google" || account?.provider ===
         "github") {
        const existingUser = await prisma.user.findUnique({
          where: { email: user.email }
        });

        if (!existingUser) {
          // Creer l'utilisateur si premiere connexion OAuth
          await prisma.user.create({
            data: {
              email: user.email,
              name: user.name,
              image: user.image,
              emailVerified: new Date(),
              role: "USER" // Role par defaut
            }
          });
        }
      }
      return true;
    }
  },

  // Pages personnalisees
  pages: {
    signIn: "/auth/login",      // Page de connexion
    error: "/auth/error",       // Page d'erreur
    newUser: "/auth/register"   // Page d'inscription
  },

  // Configuration de la session
  session: {
    strategy: "jwt",      // Utilise JWT (pas de stockage en DB)
    maxAge: 30 * 24 * 60 * 60 // 30 jours
  },

  // Secret pour signer les tokens
  secret: process.env.NEXTAUTH_SECRET
};
```

Attention Critique

Points critiques de securite :

- **NEXTAUTH_SECRET** doit etre une chaine longue et aleatoire
- Ne jamais exposer les secrets dans le code source
- Toujours hasher les mots de passe avec bcrypt
- Verifier les erreurs et ne pas donner d'indices sur les comptes existants

Etape 6 – Creer le Route Handler NextAuth

Correction Complete

Creez le fichier `app/api/auth/[...nextauth]/route.js` :

```
/*
  app/api/auth/[...nextauth]/route.js
  Route Handler pour NextAuth.js

  Ce fichier gere toutes les routes d'authentification :
  - /api/auth/signin : connexion
  - /api/auth/signout : deconnexion
  - /api/auth/callback/* : callbacks OAuth
  - /api/auth/session : recuperation session
*/

import NextAuth from "next-auth";
import { authOptions } from "@/lib/auth";

// Creation du handler NextAuth
const handler = NextAuth(authOptions);

// Export pour les methodes GET et POST
export { handler as GET, handler as POST };
```

Explication Detaillee

Pourquoi [...nextauth] ?
La syntaxe `[...nextauth]` est une **catch-all route**. Elle capture tous les segments apres `/api/auth/`.
Exemples :

- `/api/auth/signin` : page de connexion
- `/api/auth/signout` : deconnexion
- `/api/auth/callback/google` : callback OAuth Google
- `/api/auth/session` : donnees de session

2.2 Exercice 2 : Pages d'Inscription et de Connexion (45 min)

Exercice 2 : Formulaires d'authentification (45 min)

But : Creer les pages d'inscription et de connexion pour notre e-commerce.

Etape 1 – Creer l'API d'inscription

Correction Complete

Creez le fichier `app/api/auth/register/route.js` :

```
/*
  app/api/auth/register/route.js
  API pour l'inscription des utilisateurs

  Etapes :
  1. Valider les donnees recues
  2. Verifier si l'email existe deja
  3. Hasher le mot de passe
  4. Creer l'utilisateur en base
  5. Retourner une reponse de succes
*/

import { NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";
import bcrypt from "bcryptjs";

export async function POST(request) {
  try {
    // Recuperer les donnees du formulaire
    const body = await request.json();
    const { name, email, password } = body;

    // Validation des champs obligatoires
    if (!email || !password) {
      return NextResponse.json(
        { message: "Email et mot de passe requis" },
        { status: 400 }
      );
    }

    // Validation du format email (simple)
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
      return NextResponse.json(
        { message: "Format email invalide" },
        { status: 400 }
      );
    }

    // Validation longueur mot de passe
    if (password.length < 6) {
      return NextResponse.json(
        { message: "Mot de passe trop court (minimum 6 caracteres)" },
        { status: 400 }
      );
    }

    // Verifier si l'email existe deja
    const existingUser = await prisma.user.findUnique({
      where: { email: email.toLowerCase() }
```

```
    });

    if (existingUser) {
      return NextResponse.json(
        { message: "Cet email est deja utilise" },
        { status: 400 }
      );
    }

    // Hasher le mot de passe avec bcrypt
    // Le "10" est le nombre de rounds (cout de calcul)
    const hashedPassword = await bcrypt.hash(password, 10);

    // Creer l'utilisateur en base
    const user = await prisma.user.create({
      data: {
        name: name?.trim() || null,
        email: email.toLowerCase(),
        password: hashedPassword,
        role: "USER" // Role par defaut
      }
    });

    // Retourner une reponse de succes (sans le mot de passe !)
    return NextResponse.json({
      message: "Inscription reussie",
      user: {
        id: user.id,
        name: user.name,
        email: user.email
      }
    }, { status: 201 });

  } catch (error) {
    console.error("Erreur inscription:", error);
    return NextResponse.json(
      { message: "Erreur serveur lors de l'inscription" },
      { status: 500 }
    );
  }
}
```

Attention Critique

Bonnes pratiques de securite :

- Toujours hasher les mots de passe AVANT de les stocker
- Ne jamais renvoyer le mot de passe dans la reponse
- Normaliser les emails en minuscules
- Valider les entrees cote serveur (meme si fait cote client)

Etape 2 – Page d'inscription

Correction Complete

Creez le fichier `app/auth/register/page.js` :

```
/*
  app/auth/register/page.js
  Page d'inscription utilisateur

  Composant Client car utilise :
  - useState pour les champs du formulaire
  - useRouter pour la redirection
  - Evenements onClick/onChange
*/

"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";

export default function RegisterPage() {
  // Etats pour les champs du formulaire
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");

  // Etats pour le feedback
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  // Hook pour la navigation
  const router = useRouter();

  // Fonction de soumission du formulaire
  async function handleSubmit(e) {
    // Empecher le rechargement de la page
    e.preventDefault();

    // Reinitialiser l'erreur
    setError("");

    // Validation cote client
    if (!email || !password) {
      setError("Email et mot de passe requis");
      return;
    }

    if (password !== confirmPassword) {
      setError("Les mots de passe ne correspondent pas");
      return;
    }

    if (password.length < 6) {
      setError("Mot de passe trop court (minimum 6 caracteres)");
```

```
      return;
    }

    // Demarrer le chargement
    setLoading(true);

    try {
      // Appeler l'API d'inscription
      const response = await fetch("/api/auth/register", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name, email, password })
      });

      const data = await response.json();

      if (!response.ok) {
        // Afficher l'erreur du serveur
        setError(data.message || "Erreur lors de l'inscription");
        return;
      }

      // Inscription reussie : redirection vers la connexion
      router.push("/auth/login?registered=true");

    } catch (err) {
      setError("Erreur de connexion au serveur");
    } finally {
      setLoading(false);
    }
  }

  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-100 px-4">
      <div className="max-w-md w-full bg-white rounded-lg shadow-md p-8">
        {/* Titre */}
        <h1 className="text-2xl font-bold text-center text-gray-800 mb-6">
          Creer un compte
        </h1>

        {/* Message d'erreur */}
        {error && (
          <div className="bg-red-50 border border-red-200 text-red-600 px-4 py-3 rounded mb-4">
            {error}
          </div>
        )}

        {/* Formulaire */}
        <form onSubmit={handleSubmit} className="space-y-4">
          {/* Champ Nom */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">
              Nom (optionnel)
```

```
            </label>
            <input
              type="text"
              value={name}
              onChange={(e) => setName(e.target.value)}
              className="w-full border border-gray-300 rounded-md px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              placeholder="Votre nom"
            />
          </div>

          {/* Champ Email */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">
              Email *
            </label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="w-full border border-gray-300 rounded-md px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              placeholder="votre@email.com"
              required
            />
          </div>

          {/* Champ Mot de passe */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">
              Mot de passe *
            </label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="w-full border border-gray-300 rounded-md px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              placeholder="Minimum 6 caracteres"
              required
            />
          </div>

          {/* Champ Confirmation */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">
              Confirmer le mot de passe *
            </label>
            <input
              type="password"
              value={confirmPassword}
              onChange={(e) => setConfirmPassword(e.target.value)}
```

```
154                 className="w-full border border-gray-300 rounded-md
                       px-3 py-2 focus:outline-none focus:ring-2
                       focus:ring-blue-500"
155                 placeholder="Retapez votre mot de passe"
156                 required
157               />
158             </div>
159
160             {/* Bouton de soumission */}
161             <button
162               type="submit"
163               disabled={loading}
164               className="w-full bg-blue-600 text-white py-2 rounded-md
                     hover:bg-blue-700 transition disabled:opacity-50
                     disabled:cursor-not-allowed"
165             >
166               {loading ? "Inscription en cours..." : "S'inscrire"}
167             </button>
168           </form>
169
170           {/* Lien vers connexion */}
171           <p className="mt-6 text-center text-gray-600">
172             Deja un compte ?{" "}
173             <Link href="/auth/login" className="text-blue-600
                   hover:underline">
174               Se connecter
175             </Link>
176           </p>
177         </div>
178       </div>
179     );
180   }
```

Etape 3 – Page de connexion

Correction Complete

Creez le fichier `app/auth/login/page.js` :

```
/*
  app/auth/login/page.js
  Page de connexion utilisateur

  Utilise signIn de next-auth/react pour :
  - Connexion email/mot de passe (credentials)
  - Connexion OAuth (Google, GitHub)
*/

"use client";

import { useState, useEffect } from "react";
import { signIn } from "next-auth/react";
import { useRouter, useSearchParams } from "next/navigation";
import Link from "next/link";

export default function LoginPage() {
  // Etats pour le formulaire
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const [loading, setLoading] = useState(false);

  // Hooks de navigation
  const router = useRouter();
  const searchParams = useSearchParams();

  // Verifier si l'utilisateur vient de s'inscrire
  useEffect(() => {
    if (searchParams.get("registered") === "true") {
      setSuccess("Inscription reussie ! Vous pouvez maintenant vous connecter.");
    }
  }, [searchParams]);

  // Fonction de connexion email/mot de passe
  async function handleCredentialsLogin(e) {
    e.preventDefault();
    setError("");
    setLoading(true);

    try {
      // Appeler signIn de NextAuth avec le provider "credentials"
      const result = await signIn("credentials", {
        email,
        password,
        redirect: false // Gerer la redirection manuellement
      });

      if (result?.error) {
```

```
        setError(result.error);
      } else {
        // Connexion reussie : redirection vers l'accueil
        router.push("/");
        router.refresh(); // Rafraichir pour mettre a jour la session
      }
    } catch (err) {
      setError("Erreur de connexion");
    } finally {
      setLoading(false);
    }
  }

  // Fonction de connexion OAuth
  function handleOAuthLogin(provider) {
    signIn(provider, { callbackUrl: "/" });
  }

  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-100 px-4">
      <div className="max-w-md w-full bg-white rounded-lg shadow-md p-8">
        {/* Titre */}
        <h1 className="text-2xl font-bold text-center text-gray-800 mb-6">
          Connexion
        </h1>

        {/* Message de succes */}
        {success && (
          <div className="bg-green-50 border border-green-200 text-green-600 px-4 py-3 rounded mb-4">
            {success}
          </div>
        )}

        {/* Message d'erreur */}
        {error && (
          <div className="bg-red-50 border border-red-200 text-red-600 px-4 py-3 rounded mb-4">
            {error}
          </div>
        )}

        {/* Formulaire email/mot de passe */}
        <form onSubmit={handleCredentialsLogin} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">
              Email
            </label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="w-full border border-gray-300 rounded-md
```

```
                px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              placeholder="votre@email.com"
              required
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">
              Mot de passe
            </label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="w-full border border-gray-300 rounded-md px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              placeholder="Votre mot de passe"
              required
            />
          </div>

          <button
            type="submit"
            disabled={loading}
            className="w-full bg-blue-600 text-white py-2 rounded-md hover:bg-blue-700 transition disabled:opacity-50"
          >
            {loading ? "Connexion..." : "Se connecter"}
          </button>
        </form>

        {/* Separateur */}
        <div className="my-6 flex items-center">
          <div className="flex-1 border-t border-gray-300"></div>
          <span className="px-4 text-gray-500 text-sm">ou</span>
          <div className="flex-1 border-t border-gray-300"></div>
        </div>

        {/* Boutons OAuth */}
        <div className="space-y-3">
          <button
            onClick={() => handleOAuthLogin("google")}
            className="w-full flex items-center justify-center gap-2 border border-gray-300 py-2 rounded-md hover:bg-gray-50 transition"
          >
            <svg className="w-5 h-5" viewBox="0 0 24 24">
              <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
              <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
```

```
              <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
              <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
            </svg>
            Continuer avec Google
          </button>

          <button
            onClick={() => handleOAuthLogin("github")}
            className="w-full flex items-center justify-center gap-2 border border-gray-300 py-2 rounded-md hover:bg-gray-50 transition"
          >
            <svg className="w-5 h-5" fill="currentColor" viewBox="0 0 24 24">
              <path d="M12 0c-6.626 0-12 5.373-12 12 0 5.302 3.438 9.8 8.207 11.387.599.111.793-.261.793-.577v-2.234c-3.338.726-4.033-1.416-4 1.205.084 1.839 1.237 1.839 1.237 1.07 1.834 2.807 1.304 3.492.997.107-.775.418-1.305.762-1.604-2.665-.305-5.467-1.334-5.0-1.311.469-2.381 1.236-3.221-.124-.303-.535-1.524.117-3.176 0 0 1.008-.322 3.301 1.23.957-.266 1.983-.399 3.003-.404 1.02.005 2.047.138 3.006.404 2.291-1.552 3.297-1.23 3.297-1.23.653 1.653.242 2.874.118 3.176.77.84 1.235 1.911 1.235 3.221 0 4.609-2.807 5.624-5.479 5.921.43.372.823 1.102.823 2.222v3.293c0 .319.192.694.801.576 4.765-1.589 8.199-6.086 8.199-11.386 0-6.627-5.373-12-12-12z"/>
            </svg>
            Continuer avec GitHub
          </button>
        </div>

        {/* Lien vers inscription */}
        <p className="mt-6 text-center text-gray-600">
          Pas encore de compte ?{" "}
          <Link href="/auth/register" className="text-blue-600 hover:underline">
            S'inscrire
          </Link>
        </p>
      </div>
    </div>
  );
}
```

Etape 4 – Composant Header avec session

Correction Complete

Modifiez le fichier `src/components/Header.js` pour afficher l'etat de connexion :

```
/*
  src/components/Header.js
  Header avec gestion de la session utilisateur

  Affiche :
  - Liens de navigation
  - Bouton connexion/deconnexion selon l'etat
  - Avatar et nom de l'utilisateur connecte
*/

"use client";

import Link from "next/link";
import { useSession, signOut } from "next-auth/react";

export default function Header() {
  // Hook pour recuperer la session
  // status peut etre : "loading", "authenticated", "unauthenticated"
  const { data: session, status } = useSession();

  return (
    <header className="bg-white shadow-sm">
      <div className="max-w-7xl mx-auto px-4 py-4">
        <div className="flex items-center justify-between">
          {/* Logo */}
          <Link href="/" className="text-xl font-bold text-blue-600">
            My Shop
          </Link>

          {/* Navigation */}
          <nav className="flex items-center gap-6">
            <Link href="/products" className="text-gray-600
                hover:text-blue-600">
              Produits
            </Link>
            <Link href="/cart" className="text-gray-600
                hover:text-blue-600">
              Panier
            </Link>

            {/* Affichage conditionnel selon l'etat de connexion */}
            {status === "loading" ? (
              // Chargement de la session
              <span className="text-gray-400">...</span>
            ) : session ? (
              // Utilisateur connecte
              <div className="flex items-center gap-4">
                {/* Lien admin si role ADMIN */}
                {session.user.role === "ADMIN" && (
                  <Link href="/admin" className="text-purple-600
                      hover:underline">
```

```
                    Admin
                  </Link>
                )}

                {/* Avatar et nom */}
                <div className="flex items-center gap-2">
                  {session.user.image ? (
                    <img
                      src={session.user.image}
                      alt="Avatar"
                      className="w-8 h-8 rounded-full"
                    />
                  ) : (
                    <div className="w-8 h-8 rounded-full bg-blue-100 flex items-center justify-center">
                      <span className="text-blue-600 font-medium">
                        {session.user.name?.[0] || session.user.email?.[0]}
                      </span>
                    </div>
                  )}
                  <span className="text-gray-700">
                    {session.user.name || session.user.email}
                  </span>
                </div>

                {/* Bouton deconnexion */}
                <button
                  onClick={() => signOut({ callbackUrl: "/" })}
                  className="text-red-600 hover:underline"
                >
                  Deconnexion
                </button>
              </div>
            ) : (
              // Utilisateur non connecte
              <div className="flex items-center gap-4">
                <Link href="/auth/login" className="text-gray-600 hover:text-blue-600">
                  Connexion
                </Link>
                <Link
                  href="/auth/register"
                  className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700"
                >
                  Inscription
                </Link>
              </div>
            )}
          </nav>
        </div>
      </div>
    </header>
  );
}
```

Etape 5 – SessionProvider dans le layout

Correction Complete

Pour que le hook `useSession` fonctionne, il faut envelopper l'application avec `SessionProvider`.
Creez le fichier `app/providers.js` :

```
/*
  app/providers.js
  Composant Client qui fournit le contexte de session

  Ce composant est necessaire car SessionProvider
  utilise le Context React qui ne fonctionne que cote client.
*/

"use client";

import { SessionProvider } from "next-auth/react";

export default function Providers({ children }) {
  return (
    <SessionProvider>
      {children}
    </SessionProvider>
  );
}
```

Correction Complete

Modifiez `app/layout.js` pour utiliser le Provider :

```
/*
  app/layout.js
  Layout racine avec SessionProvider
*/

import "./globals.css";
import Header from "@/components/Header";
import Providers from "./providers";

export const metadata = {
  title: "My Shop - E-commerce Next.js",
  description: "Boutique en ligne avec Next.js et authentification"
};

export default function RootLayout({ children }) {
  return (
    <html lang="fr">
      <body>
        {/* SessionProvider englobe toute l'application */}
        <Providers>
          <Header />
          <main className="min-h-screen">
            {children}
```

```
            </main>
          </Providers>
        </body>
      </html>
    );
}
```

Resultat Attendu

A ce stade, vous avez une authentification fonctionnelle :

- ✔ Inscription avec email/mot de passe
- ✔ Connexion credentials et OAuth
- ✔ Session accessible partout dans l'application
- ✔ Header qui affiche l'etat de connexion

2.3 Exercice 3 : Middleware de Protection des Routes (30 min)

Exercice 3 : Middleware Next.js (30 min)

But : Proteger les routes sensibles avec un middleware.

Creer le fichier middleware.js

Correction Complete

Creez le fichier `middleware.js` **a la racine du projet** (meme niveau que `app/`) :

```
/*
  middleware.js
  Middleware de protection des routes

  Ce fichier s'execute AVANT chaque requete.
  Il permet de :
  - Rediriger les utilisateurs non connectes
  - Verifier les roles pour les zones admin
  - Proteger les routes sensibles

  IMPORTANT : Ce fichier doit etre a la racine du projet !
*/

import { NextResponse } from "next/server";
import { getToken } from "next-auth/jwt";

export async function middleware(request) {
  // Recuperer le token JWT de la session
  const token = await getToken({
    req: request,
    secret: process.env.NEXTAUTH_SECRET
  });

  // URL de la requete
  const { pathname } = request.nextUrl;

  // ================================================
  // Routes protegees : utilisateur connecte requis
  // ================================================
  const protectedRoutes = [
    "/cart",
    "/orders",
    "/profile"
  ];

  // Verifier si la route est protegee
  const isProtectedRoute = protectedRoutes.some(route =>
    pathname.startsWith(route)
  );

  // Rediriger vers login si non connecte
  if (isProtectedRoute && !token) {
    const loginUrl = new URL("/auth/login", request.url);
```

```
    // Ajouter l'URL de retour
    loginUrl.searchParams.set("callbackUrl", pathname);
    return NextResponse.redirect(loginUrl);
  }

  // ================================================
  // Routes admin : role ADMIN requis
  // ================================================
  const adminRoutes = ["/admin"];

  const isAdminRoute = adminRoutes.some(route =>
    pathname.startsWith(route)
  );

  if (isAdminRoute) {
    // Non connecte -> redirection login
    if (!token) {
      const loginUrl = new URL("/auth/login", request.url);
      loginUrl.searchParams.set("callbackUrl", pathname);
      return NextResponse.redirect(loginUrl);
    }

    // Connecte mais pas admin -> page non autorise
    if (token.role !== "ADMIN") {
      return NextResponse.redirect(new URL("/unauthorized",
          request.url));
    }
  }

  // ================================================
  // Routes auth : rediriger si deja connecte
  // ================================================
  const authRoutes = ["/auth/login", "/auth/register"];

  const isAuthRoute = authRoutes.some(route =>
    pathname.startsWith(route)
  );

  // Si connecte et sur page auth -> rediriger vers accueil
  if (isAuthRoute && token) {
    return NextResponse.redirect(new URL("/", request.url));
  }

  // Continuer la requete normalement
  return NextResponse.next();
}

// Configuration : sur quelles routes le middleware s'execute
export const config = {
  matcher: [
    /*
     * Match toutes les routes sauf :
     * - api (pas les routes API)
     * - _next/static (fichiers statiques)
     * - _next/image (optimisation images)
     * - favicon.ico
     */
    "/((?!api|_next/static|_next/image|favicon.ico).*)"
```

```
  ]
};
```

Explication Detaillee

Comment fonctionne le matcher ?
Le `matcher` definit sur quelles routes le middleware s'execute.
`"/((?!api|_next/static|_next/image|favicon.ico).*)"` signifie :

- `/(.*)` : toutes les routes
- `(?!...)` : SAUF celles qui commencent par...
- `api` : les routes API
- `_next/static` : les fichiers statiques
- `_next/image` : les images optimisees

Cela evite d'executer le middleware sur des ressources qui n'ont pas besoin de protection.

Attention Critique

Points critiques du Middleware :

- Le fichier DOIT s'appeler `middleware.js` exactement
- Il DOIT etre a la racine (pas dans `app/` ou `src/`)
- Il s'execute a l'Edge, donc pas d'acces a Prisma directement
- Utiliser `getToken` de next-auth/jwt, pas la session

Page Non Autorise

Correction Complete

Creez la page `app/unauthorized/page.js` :

```
/*
  app/unauthorized/page.js
  Page affichee quand l'utilisateur n'a pas les droits
*/

import Link from "next/link";

export default function UnauthorizedPage() {
  return (
    <div className="min-h-screen flex items-center justify-center
       bg-gray-100">
      <div className="text-center">
        <h1 className="text-6xl font-bold text-red-600 mb-4">403</h1>
        <h2 className="text-2xl font-semibold text-gray-800 mb-2">
          Acces non autorise
        </h2>
        <p className="text-gray-600 mb-6">
          Vous n'avez pas les permissions necessaires pour acceder a
             cette page.
```

```
        </p>
        <Link
          href="/"
          className="bg-blue-600 text-white px-6 py-2 rounded-md
             hover:bg-blue-700"
        >
          Retour a l'accueil
        </Link>
      </div>
    </div>
  );
}
```

2.4 Exercice 4 : Dashboard Admin avec Verification des Roles (45 min)

Exercice 4 : Dashboard Admin (45 min)

But : Creer un dashboard admin accessible uniquement aux utilisateurs avec le role ADMIN.

Etape 1 – Helper de verification session (Server Component)

Correction Complete

Creez `src/lib/auth-utils.js` pour les verifications cote serveur :

```
/*
  src/lib/auth-utils.js
  Utilitaires d'authentification pour les Server Components

  Ces fonctions permettent de :
  - Verifier si l'utilisateur est connecte
  - Verifier si l'utilisateur a un role specifique
  - Recuperer la session cote serveur
*/

import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/auth";
import { redirect } from "next/navigation";

// Recuperer la session cote serveur
export async function getSession() {
  return await getServerSession(authOptions);
}

// Verifier que l'utilisateur est connecte
// Redirige vers login si non connecte
export async function requireAuth() {
  const session = await getSession();

  if (!session) {
    redirect("/auth/login");
  }

  return session;
}

// Verifier que l'utilisateur a le role ADMIN
// Redirige vers unauthorized si pas admin
export async function requireAdmin() {
  const session = await requireAuth();

  if (session.user.role !== "ADMIN") {
    redirect("/unauthorized");
  }

  return session;
```

```
}

// Verifier un role specifique
export async function requireRole(role) {
  const session = await requireAuth();

  if (session.user.role !== role) {
    redirect("/unauthorized");
  }

  return session;
}
```

Etape 2 – Layout Admin

Correction Complete

Creez `app/admin/layout.js` :

```
/*
  app/admin/layout.js
  Layout pour la section admin

  Ce layout :
  - Verifie que l'utilisateur est admin
  - Ajoute une sidebar de navigation admin
  - Affiche un style specifique
*/

import Link from "next/link";
import { requireAdmin } from "@/lib/auth-utils";

export default async function AdminLayout({ children }) {
  // Verification du role admin (redirige si pas autorise)
  const session = await requireAdmin();

  return (
    <div className="flex min-h-screen bg-gray-100">
      {/* Sidebar */}
      <aside className="w-64 bg-gray-800 text-white">
        <div className="p-4">
          <h2 className="text-xl font-bold mb-6">Admin Panel</h2>
          <p className="text-gray-400 text-sm mb-4">
            Connecte : {session.user.name || session.user.email}
          </p>

          <nav className="space-y-2">
            <Link
              href="/admin"
              className="block px-4 py-2 rounded hover:bg-gray-700"
            >
              Dashboard
            </Link>
            <Link
              href="/admin/products"
              className="block px-4 py-2 rounded hover:bg-gray-700"
```

```
            >
              Produits
            </Link>
            <Link
              href="/admin/categories"
              className="block px-4 py-2 rounded hover:bg-gray-700"
            >
              Categories
            </Link>
            <Link
              href="/admin/orders"
              className="block px-4 py-2 rounded hover:bg-gray-700"
            >
              Commandes
            </Link>
            <Link
              href="/admin/users"
              className="block px-4 py-2 rounded hover:bg-gray-700"
            >
              Utilisateurs
            </Link>
          </nav>
        </div>

        <div className="p-4 mt-auto">
          <Link
            href="/"
            className="block px-4 py-2 text-gray-400
              hover:text-white"
          >
            Retour au site
          </Link>
        </div>
      </aside>

      {/* Contenu principal */}
      <main className="flex-1 p-8">
        {children}
      </main>
    </div>
  );
}
```

Etape 3 – Page Dashboard Admin

Correction Complete

Creez `app/admin/page.js` :

```
/*
  app/admin/page.js
  Dashboard principal de l'administration

  Affiche des statistiques :
  - Nombre de produits
  - Nombre de commandes
  - Nombre d'utilisateurs
  - Chiffre d'affaires
*/

import { prisma } from "@/lib/prisma";
import { requireAdmin } from "@/lib/auth-utils";

export default async function AdminDashboard() {
  // Verification admin (double securite avec le layout)
  await requireAdmin();

  // Recuperer les statistiques
  const [
    productsCount,
    ordersCount,
    usersCount,
    totalRevenue
  ] = await Promise.all([
    // Nombre de produits
    prisma.product.count(),

    // Nombre de commandes
    prisma.order.count(),

    // Nombre d'utilisateurs
    prisma.user.count(),

    // Chiffre d'affaires (somme des commandes payees)
    prisma.order.aggregate({
      where: { status: "PAID" },
      _sum: { total: true }
    })
  ]);

  // Formater le chiffre d'affaires
  const revenue = totalRevenue._sum.total || 0;

  return (
    <div>
      <h1 className="text-3xl font-bold text-gray-800 mb-8">
        Dashboard Administration
      </h1>

      {/* Cartes statistiques */}
```

```
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
        {/* Produits */}
        <div className="bg-white rounded-lg shadow p-6">
          <h3 className="text-gray-500 text-sm font-medium">Produits</h3>
          <p className="text-3xl font-bold text-blue-600 mt-2">
            {productsCount}
          </p>
        </div>

        {/* Commandes */}
        <div className="bg-white rounded-lg shadow p-6">
          <h3 className="text-gray-500 text-sm font-medium">Commandes</h3>
          <p className="text-3xl font-bold text-green-600 mt-2">
            {ordersCount}
          </p>
        </div>

        {/* Utilisateurs */}
        <div className="bg-white rounded-lg shadow p-6">
          <h3 className="text-gray-500 text-sm font-medium">Utilisateurs</h3>
          <p className="text-3xl font-bold text-purple-600 mt-2">
            {usersCount}
          </p>
        </div>

        {/* Chiffre d'affaires */}
        <div className="bg-white rounded-lg shadow p-6">
          <h3 className="text-gray-500 text-sm font-medium">Chiffre d'affaires</h3>
          <p className="text-3xl font-bold text-orange-600 mt-2">
            {revenue.toFixed(2)} DT
          </p>
        </div>
      </div>

      {/* Actions rapides */}
      <div className="bg-white rounded-lg shadow p-6">
        <h2 className="text-xl font-semibold text-gray-800 mb-4">
          Actions rapides
        </h2>
        <div className="flex gap-4">
          <a
            href="/admin/products/new"
            className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700"
          >
            Ajouter un produit
          </a>
          <a
            href="/admin/categories/new"
            className="bg-green-600 text-white px-4 py-2 rounded hover:bg-green-700"
          >
            Ajouter une categorie
```

```
            </a>
          </div>
        </div>
      </div>
    );
}
```

Astuce Professionnelle

Double securite :
Meme si le Middleware protege les routes `/admin`, nous ajoutons aussi `requireAdmin()` dans le layout car :

- Defense en profondeur
- Le Middleware peut etre contourne si mal configure
- Les Server Actions ne passent pas par le Middleware

Etape 4 – Page de gestion des utilisateurs

Correction Complete

Creez `app/admin/users/page.js` :

```
/*
  app/admin/users/page.js
  Page de gestion des utilisateurs

  Permet a l'admin de :
  - Voir tous les utilisateurs
  - Voir leurs roles
  - (Future : modifier les roles)
*/

import { prisma } from "@/lib/prisma";
import { requireAdmin } from "@/lib/auth-utils";

export default async function AdminUsersPage() {
  await requireAdmin();

  // Recuperer tous les utilisateurs
  const users = await prisma.user.findMany({
    orderBy: { createdAt: "desc" },
    select: {
      id: true,
      name: true,
      email: true,
      role: true,
      createdAt: true,
      _count: {
        select: { orders: true }
      }
    }
  });

  return (
    <div>
      <h1 className="text-3xl font-bold text-gray-800 mb-8">
        Gestion des Utilisateurs
      </h1>

      {/* Tableau des utilisateurs */}
      <div className="bg-white rounded-lg shadow overflow-hidden">
        <table className="w-full">
          <thead className="bg-gray-50">
            <tr>
              <th className="px-6 py-3 text-left text-xs font-medium
                 text-gray-500 uppercase">
                Utilisateur
              </th>
              <th className="px-6 py-3 text-left text-xs font-medium
                 text-gray-500 uppercase">
                Email
              </th>
              <th className="px-6 py-3 text-left text-xs font-medium
```

```
                  text-gray-500 uppercase">
                Role
              </th>
              <th className="px-6 py-3 text-left text-xs font-medium
                  text-gray-500 uppercase">
                Commandes
              </th>
              <th className="px-6 py-3 text-left text-xs font-medium
                  text-gray-500 uppercase">
                Inscription
              </th>
            </tr>
          </thead>
          <tbody className="divide-y divide-gray-200">
            {users.map((user) => (
              <tr key={user.id}>
                <td className="px-6 py-4 whitespace-nowrap">
                  <span className="font-medium text-gray-900">
                    {user.name || "Sans nom"}
                  </span>
                </td>
                <td className="px-6 py-4 whitespace-nowrap
                    text-gray-600">
                  {user.email}
                </td>
                <td className="px-6 py-4 whitespace-nowrap">
                  <span className={
                    user.role === "ADMIN"
                      ? "px-2 py-1 bg-purple-100 text-purple-800
                          rounded text-sm"
                      : "px-2 py-1 bg-gray-100 text-gray-800 rounded
                          text-sm"
                  }>
                    {user.role}
                  </span>
                </td>
                <td className="px-6 py-4 whitespace-nowrap
                    text-gray-600">
                  {user._count.orders}
                </td>
                <td className="px-6 py-4 whitespace-nowrap
                    text-gray-600">
                  {new
                      Date(user.createdAt).toLocaleDateString("fr-FR")}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

2.5 Exercice 5 : Optimisation des Performances (60 min)

Exercice 5 : Performance et Optimisation (60 min)

But : Optimiser les performances de l'application e-commerce avec les outils Next.js.

Etape 1 – Optimisation des images avec next/image

Concept Fondamental

Pourquoi next/image ?
Le composant `Image` de Next.js offre :

- ✔ **Redimensionnement automatique** selon le device
- ✔ **Formats modernes** (WebP, AVIF) automatiquement
- ✔ **Lazy loading** natif (charge quand visible)
- ✔ **Placeholder flou** pendant le chargement
- ✔ **Prevention CLS** (reserve l'espace)

Correction Complete

Modifiez le composant `src/components/ProductCard.js` :

```
/*
  src/components/ProductCard.js
  Carte produit optimisee avec next/image

  Utilise le composant Image de Next.js pour :
  - Optimiser automatiquement les images
  - Charger en lazy loading
  - Afficher un placeholder flou
*/

import Image from "next/image";
import Link from "next/link";

export default function ProductCard({ product }) {
  return (
    <Link href={/products/}>
      <div className="bg-white rounded-lg shadow-md overflow-hidden hover:shadow-lg transition-shadow">
        {/* Image optimisee */}
        <div className="relative aspect-square">
          <Image
            src={product.imageUrl || "/placeholder-product.jpg"}
            alt={product.title}
            fill                          // Remplit le conteneur
            sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
            className="object-cover"      // Couvre le conteneur
            placeholder="blur"            // Placeholder flou
```

```
            blurDataURL="/placeholder-blur.jpg"  // Image floue
            loading="lazy"                // Chargement differe (defaut)
          />
        </div>

        {/* Informations produit */}
        <div className="p-4">
          <h3 className="font-semibold text-gray-800 truncate">
            {product.title}
          </h3>
          <p className="text-gray-600 text-sm mt-1 line-clamp-2">
            {product.description}
          </p>
          <p className="text-blue-600 font-bold mt-2">
            {product.price.toFixed(2)} DT
          </p>
        </div>
      </div>
    </Link>
  );
}
```

Explication Detaillee

Proprietes importantes de next/image :

- **fill** : L'image remplit son conteneur parent (qui doit etre `position: relative`)
- **sizes** : Indique la taille de l'image selon le viewport (optimise le telechargement)
- **placeholder="blur"** : Affiche une version floue pendant le chargement
- **priority** : Pour les images au-dessus de la ligne de flottaison (LCP)

Correction Complete

Configuration dans `next.config.js` pour les images externes :

```
/*
  next.config.js
  Configuration Next.js avec domaines d'images autorises
*/

const nextConfig = {
  images: {
    // Domaines autorises pour les images externes
    remotePatterns: [
      {
        protocol: "https",
        hostname: "fakestoreapi.com",
        pathname: "/**"
      },
      {
        protocol: "https",
        hostname: "images.unsplash.com",
        pathname: "/**"
```

```
      },
      {
        protocol: "https",
        hostname: "**.googleusercontent.com",
        pathname: "/**"
      }
    ],
    // Formats d'image a generer
    formats: ["image/avif", "image/webp"]
  }
};

module.exports = nextConfig;
```

Etape 2 – Optimisation des polices avec next/font

Correction Complete

Modifiez `app/layout.js` pour utiliser next/font :

```
/*
  app/layout.js
  Layout avec polices optimisees

  next/font offre :
  - Hebergement local automatique (pas de requete externe)
  - Preconnect automatique
  - Pas de FOUT (Flash of Unstyled Text)
  - Variable CSS pour utilisation flexible
*/

import "./globals.css";
import { Inter, Poppins } from "next/font/google";
import Header from "@/components/Header";
import Providers from "./providers";

// Police principale : Inter
const inter = Inter({
  subsets: ["latin"],
  display: "swap",        // Affiche le texte immediatement
  variable: "--font-inter" // Variable CSS pour Tailwind
});

// Police secondaire : Poppins pour les titres
const poppins = Poppins({
  subsets: ["latin"],
  weight: ["400", "600", "700"],
  display: "swap",
  variable: "--font-poppins"
});

export const metadata = {
  title: "My Shop - E-commerce Next.js",
  description: "Boutique en ligne avec Next.js, authentification et
      performances optimisees"
};

export default function RootLayout({ children }) {
  return (
    <html lang="fr" className={${inter.variable} }>
      <body className="font-sans">
        <Providers>
          <Header />
          <main className="min-h-screen">
            {children}
          </main>
        </Providers>
      </body>
    </html>
  );
}
```

Correction Complete

Mise a jour de `tailwind.config.js` pour utiliser les polices :

```
/*
  tailwind.config.js
  Configuration Tailwind avec polices personnalisees
*/

module.exports = {
  content: [
    "./app/**/*.{js,jsx}",
    "./src/**/*.{js,jsx}"
  ],
  theme: {
    extend: {
      fontFamily: {
        // Utilise les variables CSS definies par next/font
        sans: ["var(--font-inter)", "system-ui", "sans-serif"],
        heading: ["var(--font-poppins)", "system-ui", "sans-serif"]
      }
    }
  },
  plugins: []
};
```

Etape 3 – Lazy Loading des composants avec next/dynamic

Concept Fondamental

Quand utiliser next/dynamic ?

- Composants lourds non critiques (editeurs, graphiques)
- Composants utilises conditionnellement
- Bibliotheques externes volumineuses
- Modales et popups

Correction Complete

Exemple avec un composant de graphique :

```
/*
  app/admin/page.js (extrait)
  Chargement dynamique d'un composant de graphique
*/

import dynamic from "next/dynamic";

// Charger le composant AdminChart seulement quand necessaire
// ssr: false car les graphiques ont souvent besoin du DOM
const AdminChart = dynamic(
  () => import("@/components/AdminChart"),
  {
    loading: () => (
      <div className="h-64 bg-gray-100 animate-pulse
          rounded-lg"></div>
    ),
    ssr: false  // Ne pas rendre cote serveur
  }
);

export default async function AdminDashboard() {
  // ... code existant ...

  return (
    <div>
      {/* ... statistiques ... */}

      {/* Graphique charge dynamiquement */}
      <div className="bg-white rounded-lg shadow p-6 mt-8">
        <h2 className="text-xl font-semibold mb-4">
          Evolution des ventes
        </h2>
        <AdminChart />
      </div>
    </div>
  );
}
```

Explication Detaillee

Options de next/dynamic :

- **loading** : Composant affiche pendant le chargement
- **ssr : false** : Desactive le rendu serveur (pour composants client-only)
- Le composant n'est telecharge que quand necessaire
- Reduit le bundle JavaScript initial

Etape 4 – Analyse avec Lighthouse

Performance et Optimisation

Lancer une analyse Lighthouse :

1. Ouvrir Chrome DevTools (F12)
2. Onglet "Lighthouse"
3. Selectionner : Performance, Accessibility, Best Practices, SEO
4. Mode : Navigation, Device : Mobile
5. Cliquer sur "Analyze page load"

Objectifs pour notre e-commerce :

- Performance : > 90
- Accessibility : > 90
- Best Practices : > 90
- SEO : > 90

Correction Complete

Ajout des metadonnees SEO dans les pages produits :

```
/*
  app/products/[slug]/page.js
  Page produit avec metadonnees dynamiques
*/

import { prisma } from "@/lib/prisma";
import { notFound } from "next/navigation";
import Image from "next/image";
import AddToCartButton from "@/components/AddToCartButton";

// Generer les metadonnees dynamiquement
export async function generateMetadata({ params }) {
  const product = await prisma.product.findUnique({
    where: { slug: params.slug }
  });

  if (!product) {
    return { title: "Produit non trouve" };
  }

  return {
    title: ${product.title} - My Shop,
    description: product.description.substring(0, 160),
    openGraph: {
      title: product.title,
      description: product.description,
      images: [{ url: product.imageUrl }]
    }
  };
}

```

```
export default async function ProductPage({ params }) {
  const product = await prisma.product.findUnique({
    where: { slug: params.slug },
    include: { category: true }
  });

  if (!product) {
    notFound();
  }

  return (
    <div className="max-w-7xl mx-auto px-4 py-8">
      <div className="grid md:grid-cols-2 gap-8">
        {/* Image produit optimisee */}
        <div className="relative aspect-square">
          <Image
            src={product.imageUrl || "/placeholder-product.jpg"}
            alt={product.title}
            fill
            sizes="(max-width: 768px) 100vw, 50vw"
            className="object-cover rounded-lg"
            priority  // Image importante pour LCP
          />
        </div>

        {/* Details produit */}
        <div>
          <h1 className="text-3xl font-bold text-gray-800">
            {product.title}
          </h1>

          {product.category && (
            <p className="text-gray-500 mt-2">
              Categorie : {product.category.name}
            </p>
          )}

          <p className="text-3xl font-bold text-blue-600 mt-4">
            {product.price.toFixed(2)} DT
          </p>

          <p className="text-gray-600 mt-4">
            {product.description}
          </p>

          <div className="mt-6">
            <AddToCartButton productId={product.id} />
          </div>
        </div>
      </div>
    </div>
  );
}
```

Etape 5 – Composant AddToCart lie a l'utilisateur

Correction Complete

Creez `src/components/AddToCartButton.js` :

```
/*
  src/components/AddToCartButton.js
  Bouton d'ajout au panier avec verification de session

  Ce composant :
  - Verifie si l'utilisateur est connecte
  - Ajoute le produit au panier via Server Action
  - Affiche un feedback visuel
*/

"use client";

import { useState } from "react";
import { useSession } from "next-auth/react";
import { useRouter } from "next/navigation";
import { addToCart } from "@/app/actions/cart";

export default function AddToCartButton({ productId }) {
  const { data: session, status } = useSession();
  const router = useRouter();
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState("");

  async function handleAddToCart() {
    // Verifier si connecte
    if (status === "unauthenticated") {
      router.push("/auth/login?callbackUrl=" +
          window.location.pathname);
      return;
    }

    setLoading(true);
    setMessage("");

    try {
      // Appeler la Server Action
      const result = await addToCart(productId, 1);

      if (result.success) {
        setMessage("Ajoute au panier !");
        // Effacer le message apres 2 secondes
        setTimeout(() => setMessage(""), 2000);
      } else {
        setMessage(result.error || "Erreur");
      }
    } catch (error) {
      setMessage("Erreur lors de l'ajout");
    } finally {
      setLoading(false);
    }
  }
```

```

  return (
    <div>
      <button
        onClick={handleAddToCart}
        disabled={loading || status === "loading"}
        className="w-full bg-blue-600 text-white py-3 px-6
           rounded-lg font-semibold hover:bg-blue-700 transition
           disabled:opacity-50"
      >
        {loading ? "Ajout en cours..." : "Ajouter au panier"}
      </button>

      {message && (
        <p className="text-green-600 text-center mt-2">
          {message}
        </p>
      )}
    </div>
  );
}
```

Etape 6 – Server Action pour le panier

Correction Complete

Creez `app/actions/cart.js` :

```
/*
  app/actions/cart.js
  Server Actions pour la gestion du panier

  Les Server Actions permettent :
  - D'executer du code serveur depuis un composant client
  - De muter les donnees en base directement
  - De revalider le cache automatiquement
*/

"use server";

import { getSession } from "@/lib/auth-utils";
import { prisma } from "@/lib/prisma";
import { revalidatePath } from "next/cache";

// Ajouter un produit au panier
export async function addToCart(productId, quantity = 1) {
  // Verifier la session
  const session = await getSession();

  if (!session?.user?.id) {
    return { success: false, error: "Non connecte" };
  }

  const userId = session.user.id;

  try {
```

```
    // Verifier que le produit existe
    const product = await prisma.product.findUnique({
      where: { id: productId }
    });

    if (!product) {
      return { success: false, error: "Produit introuvable" };
    }

    // Verifier le stock
    if (product.stock < quantity) {
      return { success: false, error: "Stock insuffisant" };
    }

    // Ajouter ou mettre a jour le panier
    // upsert = update si existe, create sinon
    await prisma.cartItem.upsert({
      where: {
        userId_productId: { userId, productId }
      },
      update: {
        // Incrementer la quantite
        quantity: { increment: quantity }
      },
      create: {
        userId,
        productId,
        quantity
      }
    });

    // Revalider la page panier
    revalidatePath("/cart");

    return { success: true };

  } catch (error) {
    console.error("Erreur addToCart:", error);
    return { success: false, error: "Erreur serveur" };
  }
}

// Modifier la quantite d'un item
export async function updateCartItem(cartItemId, quantity) {
  const session = await getSession();

  if (!session?.user?.id) {
    return { success: false, error: "Non connecte" };
  }

  try {
    if (quantity <= 0) {
      // Supprimer si quantite <= 0
      await prisma.cartItem.delete({
        where: { id: cartItemId }
      });
    } else {
      // Mettre a jour la quantite
```

```
      await prisma.cartItem.update({
        where: { id: cartItemId },
        data: { quantity }
      });
    }

    revalidatePath("/cart");
    return { success: true };

  } catch (error) {
    console.error("Erreur updateCartItem:", error);
    return { success: false, error: "Erreur serveur" };
  }
}

// Supprimer un item du panier
export async function removeFromCart(cartItemId) {
  const session = await getSession();

  if (!session?.user?.id) {
    return { success: false, error: "Non connecte" };
  }

  try {
    await prisma.cartItem.delete({
      where: { id: cartItemId }
    });

    revalidatePath("/cart");
    return { success: true };

  } catch (error) {
    console.error("Erreur removeFromCart:", error);
    return { success: false, error: "Erreur serveur" };
  }
}

// Vider le panier
export async function clearCart() {
  const session = await getSession();

  if (!session?.user?.id) {
    return { success: false, error: "Non connecte" };
  }

  try {
    await prisma.cartItem.deleteMany({
      where: { userId: session.user.id }
    });

    revalidatePath("/cart");
    return { success: true };

  } catch (error) {
    console.error("Erreur clearCart:", error);
    return { success: false, error: "Erreur serveur" };
  }
}
```

2.6 Exercice 6 : Page Panier Complete (30 min)

Exercice 6 : Page Panier (30 min)

But : Creer la page panier avec gestion des quantites.

Correction Complete

Creez `app/cart/page.js` :

```
/*
  app/cart/page.js
  Page du panier utilisateur

  Affiche :
  - Liste des produits dans le panier
  - Quantites modifiables
  - Total du panier
  - Bouton pour passer commande
*/

import { requireAuth } from "@/lib/auth-utils";
import { prisma } from "@/lib/prisma";
import Image from "next/image";
import CartItemRow from "@/components/CartItemRow";
import Link from "next/link";

export default async function CartPage() {
  // Verifier que l'utilisateur est connecte
  const session = await requireAuth();

  // Recuperer les items du panier
  const cartItems = await prisma.cartItem.findMany({
    where: { userId: session.user.id },
    include: {
      product: true
    },
    orderBy: { createdAt: "desc" }
  });

  // Calculer le total
  const total = cartItems.reduce((sum, item) => {
    return sum + (item.product.price * item.quantity);
  }, 0);

  return (
    <div className="max-w-4xl mx-auto px-4 py-8">
      <h1 className="text-3xl font-bold text-gray-800 mb-8">
        Mon Panier
      </h1>

      {cartItems.length === 0 ? (
        // Panier vide
        <div className="text-center py-12">
          <p className="text-gray-600 mb-4">
            Votre panier est vide
```

```
          </p>
          <Link
            href="/products"
            className="text-blue-600 hover:underline"
          >
            Continuer mes achats
          </Link>
        </div>
      ) : (
        <div>
          {/* Liste des produits */}
          <div className="space-y-4 mb-8">
            {cartItems.map((item) => (
              <CartItemRow key={item.id} item={item} />
            ))}
          </div>

          {/* Total et bouton commande */}
          <div className="border-t pt-6">
            <div className="flex justify-between items-center mb-6">
              <span className="text-xl font-semibold">Total :</span>
              <span className="text-2xl font-bold text-blue-600">
                {total.toFixed(2)} DT
              </span>
            </div>

            <div className="flex gap-4">
              <Link
                href="/products"
                className="flex-1 text-center border border-gray-300 py-3 rounded-lg hover:bg-gray-50"
              >
                Continuer mes achats
              </Link>
              <Link
                href="/checkout"
                className="flex-1 text-center bg-blue-600 text-white py-3 rounded-lg hover:bg-blue-700"
              >
                Passer commande
              </Link>
            </div>
          </div>
        </div>
      )}
    </div>
  );
}
```

Correction Complete

Creez `src/components/CartItemRow.js` :

```
/*
  src/components/CartItemRow.js
  Ligne d'un item dans le panier

  Permet de :
  - Voir le produit et son prix
  - Modifier la quantite
  - Supprimer du panier
*/

"use client";

import { useState } from "react";
import Image from "next/image";
import { updateCartItem, removeFromCart } from "@/app/actions/cart";

export default function CartItemRow({ item }) {
  const [quantity, setQuantity] = useState(item.quantity);
  const [loading, setLoading] = useState(false);

  // Modifier la quantite
  async function handleQuantityChange(newQuantity) {
    setLoading(true);
    setQuantity(newQuantity);

    await updateCartItem(item.id, newQuantity);
    setLoading(false);
  }

  // Supprimer l'item
  async function handleRemove() {
    setLoading(true);
    await removeFromCart(item.id);
  }

  const subtotal = item.product.price * quantity;

  return (
    <div className="flex items-center gap-4 bg-white p-4 rounded-lg
        shadow">
      {/* Image produit */}
      <div className="relative w-20 h-20 flex-shrink-0">
        <Image
          src={item.product.imageUrl || "/placeholder-product.jpg"}
          alt={item.product.title}
          fill
          className="object-cover rounded"
        />
      </div>

      {/* Infos produit */}
      <div className="flex-grow">
        <h3 className="font-semibold text-gray-800">
```

```
            {item.product.title}
          </h3>
          <p className="text-gray-600">
            {item.product.price.toFixed(2)} DT
          </p>
        </div>

        {/* Quantite */}
        <div className="flex items-center gap-2">
          <button
            onClick={() => handleQuantityChange(quantity - 1)}
            disabled={loading || quantity <= 1}
            className="w-8 h-8 border rounded hover:bg-gray-100
               disabled:opacity-50"
          >
            -
          </button>
          <span className="w-8 text-center">{quantity}</span>
          <button
            onClick={() => handleQuantityChange(quantity + 1)}
            disabled={loading}
            className="w-8 h-8 border rounded hover:bg-gray-100"
          >
            +
          </button>
        </div>

        {/* Sous-total */}
        <div className="w-24 text-right font-semibold">
          {subtotal.toFixed(2)} DT
        </div>

        {/* Supprimer */}
        <button
          onClick={handleRemove}
          disabled={loading}
          className="text-red-600 hover:text-red-800"
        >
          Supprimer
        </button>
      </div>
    );
}
```

3 Recapitulatif et Structure Finale du Projet

Structure des Fichiers

Structure complete du projet apres la seance 4 :

```
my-shop/
|-- app/
|   |-- api/
|   |   |-- auth/
|   |   |   |-- [...nextauth]/
|   |   |   |   |-- route.js          # Handler NextAuth
|   |   |   |-- register/
|   |   |       |-- route.js          # API inscription
|   |   |-- products/
|   |   |   |-- route.js              # GET/POST produits
|   |   |   |-- [id]/
|   |   |       |-- route.js          # GET/PUT/DELETE produit
|   |   |-- categories/
|   |       |-- route.js
|   |
|   |-- auth/
|   |   |-- login/
|   |   |   |-- page.js               # Page connexion
|   |   |-- register/
|   |   |   |-- page.js               # Page inscription
|   |   |-- error/
|   |       |-- page.js               # Page erreur
|   |
|   |-- admin/
|   |   |-- layout.js                 # Layout admin avec sidebar
|   |   |-- page.js                   # Dashboard admin
|   |   |-- products/
|   |   |   |-- page.js               # Liste produits admin
|   |   |   |-- new/
|   |   |   |   |-- page.js           # Ajout produit
|   |   |   |-- [id]/
|   |   |       |-- page.js           # Edition produit
|   |   |-- categories/
|   |   |   |-- page.js
|   |   |-- orders/
|   |   |   |-- page.js
|   |   |-- users/
|   |       |-- page.js               # Gestion utilisateurs
|   |
|   |-- cart/
|   |   |-- page.js                   # Page panier
|   |
|   |-- products/
```

```
|   |   |-- page.js                   # Liste produits
|   |   |-- [slug]/
|   |       |-- page.js               # Detail produit
|   |
|   |-- unauthorized/
|   |   |-- page.js                   # Page 403
|   |
|   |-- actions/
|   |   |-- cart.js                   # Server Actions panier
|   |
|   |-- layout.js                     # Layout racine
|   |-- page.js                       # Accueil
|   |-- providers.js                  # SessionProvider
|   |-- globals.css
|
|-- src/
|   |-- lib/
|   |   |-- prisma.js                 # Client Prisma singleton
|   |   |-- auth.js                   # Config NextAuth
|   |   |-- auth-utils.js             # Helpers auth serveur
|   |
|   |-- components/
|       |-- Header.js                 # Header avec session
|       |-- ProductCard.js            # Carte produit optimisee
|       |-- AddToCartButton.js        # Bouton ajout panier
|       |-- CartItemRow.js            # Ligne panier
|
|-- prisma/
|   |-- schema.prisma                 # Schema avec auth
|   |-- migrations/
|
|-- middleware.js                     # Middleware protection routes
|-- next.config.js                    # Config images/fonts
|-- tailwind.config.js
|-- .env                              # Variables environnement
|-- package.json
```

4 Livrable de la Seance

Objectifs Pedagogiques

Checklist du livrable :

- ☐ Authentification complete avec NextAuth.js
- ☐ Inscription et connexion email/mot de passe fonctionnelles
- ☐ (Optionnel) OAuth Google et/ou GitHub configure
- ☐ Middleware de protection des routes implementer
- ☐ Dashboard admin accessible uniquement aux ADMIN
- ☐ Page utilisateurs dans l'admin
- ☐ Panier lie a l'utilisateur connecte
- ☐ Images optimisees avec next/image
- ☐ Polices optimisees avec next/font
- ☐ Score Lighthouse $>$ 80 sur toutes les metriques

Astuce Professionnelle

Pour tester l'application :

1. Creer un premier compte utilisateur (role USER par defaut)
2. Modifier manuellement le role en ADMIN dans Prisma Studio :

```
npx prisma studio
```

3. Se reconnecter pour voir le lien Admin dans le header
4. Tester la protection des routes en se deconnectant

5 Conclusion

Concept Fondamental

Ce que nous avons appris dans cette seance :

1. **Authentification avec NextAuth.js**
   - Configuration des providers (Credentials, Google, GitHub)
   - Callbacks JWT et session pour personnaliser les tokens
   - Integration avec Prisma pour stocker les utilisateurs
2. **Middleware Next.js**
   - Protection des routes avant le rendu
   - Verification des roles pour les zones admin
   - Redirection des utilisateurs non autorises
3. **Gestion des roles (RBAC)**
   - Champ role dans la table User
   - Verification cote serveur avec getServerSession
   - Affichage conditionnel dans l'interface
4. **Optimisation des performances**
   - Images optimisees avec next/image
   - Polices optimisees avec next/font
   - Lazy loading avec next/dynamic
   - Analyse Lighthouse

Explication Detaillee

Preparation pour la seance 5 :
Dans la derniere seance, nous aborderons :

→ Architecture avancee et Edge Functions

→ Deploiement sur Vercel

→ Variables d'environnement en production

→ CI/CD et bonnes pratiques professionnelles

→ Monitoring et analyse des performances en production

Bonne continuation dans votre projet e-commerce !

A la prochaine seance pour le deploiement final.